LAB 5: STATIC CODE ANALYSIS

Name : Harish

SRN : PES1UG24AM369

1. Objective

To improve the quality, security, and style of the assigned Python program using static analysis tools. Pylint assesses code quality, Bandit identifies common security risks, and Flake8 checks style and formatting. Reports were generated before and after code changes.

2. Program and Tools

The assigned program, maintenance_system.py, maintains a warehouse asset registry, updates service dates, identifies overdue inspections, and writes a JSON maintenance report. The tools were installed in a Python virtual environment.

Tool	Purpose
Pylint	Code-quality diagnostics, documentation, naming, and programming-practice checks.
Bandit	Checks for common insecure Python coding patterns.
Flake8	Checks formatting and style conventions, including whitespace and line length.

3. Issue Documentation and Fixes

Line numbers refer to the original maintenance_system.py; they may differ in the corrected version.

Issue	Type	Original line(s)	Description	Fix applied
Missing module docstring	Quality	1	Module purpose was undocumented.	Added a module docstring.
Mutable default argument	Bug	10	logs=[] reuses a list across calls.	Changed to logs=None and initializes a new list.
Bare except	Quality	18	Catches exceptions too broadly.	Uses specific date-parsing exceptions.
Dictionary iteration	Style	23	Iterates keys and separately indexes values.	Uses assets.items().
Global statement	Maintainability	30	Changes a module-level variable in a function.	Changed interval handling to validate and return the value.
File encoding unspecified	Portability	34, 41	Encoding depends on environment defaults.	Specifies UTF-8.
Manual file handling	Resource management	34, 41	Files are opened and closed explicitly.	Uses with statements.
Insecure temporary file	Security	40	tempfile.mktemp() may create a race condition.	Uses NamedTemporaryFile.
Assert used for validation	Reliability	61	Assertions can be disabled in optimized execution.	Uses an explicit empty-registry check.
Function naming	Style	46	printAlerts is not snake_case.	Renamed to print_alerts.
Unused import	Style	2	logging was imported but unused.	Removed the unused import.
Missing function docstrings	Documentation	Multiple	Functions lacked purpose descriptions.	Added function docstrings.
Formatting violations	Style	Multiple	Flake8 reported spacing and line-length violations.	Adjusted spacing and wrapped long lines.
Input validation	Reliability	Registration / main	Blank IDs and non-positive intervals were accepted.	Added checks for blank IDs and positive intervals.

4. Before-and-After Results

Tool	Before	After	Outcome
Pylint	6.55/10	10.00/10	Target above 9.5 achieved.
Bandit	Two findings: one Medium, one Low	No issues identified	Both reported findings addressed.
Flake8	Multiple style violations	No output / no reported violations	Formatting findings addressed.

Evidence files: pylint_before.txt, pylint_after.txt, bandit_before.txt, bandit_after.txt, flake8_before.txt, and flake8_after.txt.

5. Reflection

5.1 What is static code analysis?

Static code analysis examines source code without executing it to identify potential bugs, security vulnerabilities, and style violations. It helps developers detect issues early.

5.2 What are the purposes of Pylint, Bandit, and Flake8?

Pylint reports code-quality and programming-practice issues. Bandit identifies common insecure coding patterns. Flake8 checks Python style and formatting, including whitespace and line length.

5.3 What improvements were achieved?

The Pylint score improved from 6.55/10 to 10.00/10. Bandit reported no issues after the changes, and Flake8 produced no reported violations. Changes included safer temporary-file handling, explicit encoding, context managers, more specific exception handling, validation, and documentation.

5.4 Why is static analysis useful?

Static analysis helps find defects and risky patterns before code is deployed. It improves readability, maintainability, consistency, and security, and provides repeatable checks after code changes.

5.5 What did this lab demonstrate?

The tools complement one another: Pylint focuses on broad code quality, Bandit focuses on security, and Flake8 focuses on style. Running them before and after changes provides measurable verification.